ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

		Скобцов Ю.А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 4

Генетическое программирование в задаче символьной регрессии
Вариант 4
по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ
СТУДЕНТ

гр. №	4332К		Могилатов С.И.
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ 4

1.1 Индивидуальное задание 4

1.2 Математическая постановка 4

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 5

2.1 Представление программы деревом 5

2.2 Инициализация и операторы 5

2.3 Безопасная интерпретация 5

3 ПРОГРАММА И РЕЗУЛЬТАТЫ 6

3.1 Реализация алгоритма 6

3.2 Результат символьной регрессии 6

3.3 Графическое представление деревьев 7

3.4 Параметрическое исследование 8

4 ВЫВОДЫ 9

5 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС 10

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 11

Приложение А. Листинг алгоритма 12

Приложение Б. Сценарий экспериментов 17

ВВЕДЕНИЕ

Генетическое программирование позволяет искать структуру программы как решение задачи оптимизации. В символьной регрессии программа представляется математическим выражением, а ее качество определяется близостью вычисленных значений к наблюдаемым данным.

Цель работы - реализовать древовидный генетический алгоритм для восстановления функции варианта 4, исследовать сходимость и влияние параметров, представить лучшие программные деревья графически и сравнить найденную формулу с исходной.

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1 Индивидуальное задание

Для варианта 4 требуется найти методом символьной регрессии функцию f(x) = x^6 - 2*x^4 + x^2 по 20 случайным точкам области x ∈ [-1, 1]. Структура программы - древовидное представление. Необходимо задать терминальное и функциональное множества, выбрать фитнесс-функцию, выполнить эволюцию, показать деревья и формулы, сравнить исходную и найденную функции, а также ответить на контрольный вопрос 4.

Таблица 1 - Параметры индивидуального задания

Показатель	Значение
Вариант	4
Целевая функция	x^6 - 2*x^4 + x^2
Область	[-1, 1]
Обучающее множество	20 случайных точек
Представление	древовидное
Контрольный вопрос	4

1.2 Математическая постановка

Пусть X = {(x_i, y_i)} - обучающее множество, где y_i = f(x_i). Каждая особь p задает вычислимую программу p(x). Требуется минимизировать среднеквадратичную ошибку MSE(p) = (1/N) sum_i (p(x_i) - y_i)^2. Для предотвращения неоправданно больших деревьев в оценку добавляется небольшой штраф, пропорциональный числу узлов.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Представление программы деревом

В древовидном представлении листья содержат терминалы: переменную x и константы, а внутренние вершины - операции из функционального множества. Арность операции определяет число дочерних узлов. Например, выражение x^6 - 2*x^4 + x^2 представляется деревом, в котором корневой оператор сложения объединяет разность x^6 - 2*x^4 и слагаемое x^2.

2.2 Инициализация и операторы

Начальная популяция создается комбинацией методов full и grow. При турнирном отборе из случайной группы выбирается особь с минимальной оценкой. Кроссовер обменивает случайные поддеревья двух родителей, а мутация заменяет случайное поддерево новым. Элитизм переносит лучшую особь в следующее поколение.

2.3 Безопасная интерпретация

В алгоритме используются защищенное деление, ограничение аргумента экспоненты и ограничение показателя степени. Нечисловые результаты и переполнение получают штраф. Операции pow2, pow4 и pow6 являются безопасными частными случаями возведения в целую степень и позволяют компактно описывать полиномы.

3 ПРОГРАММА И РЕЗУЛЬТАТЫ

3.1 Реализация алгоритма

В файле gp_symbolic_regression.py реализованы узел дерева Node, рекурсивная интерпретация, печать формулы, генерация начальной популяции, турнирный отбор, кроссовер поддеревьев, мутация и основной цикл. Сценарий run_experiments.py фиксирует seed 4, создает данные, запускает базовую серию и параметрическое исследование, сохраняет JSON/CSV и изображения.

В базовом запуске использованы популяция 500 особей, 150 поколений, максимальная глубина 7, вероятность кроссовера 0,85, вероятность мутации 0,12, размер турнира 5 и коэффициент парсимонии 10^-5.

Таблица 2 - Параметры базового запуска

Параметр	Значение
Размер популяции	500
Число поколений	150
Максимальная глубина	7
Кроссовер	0,85
Мутация	0,12
Турнир	5 особей
Seed	4

3.2 Результат символьной регрессии

В базовом запуске найдено выражение: ((sin(3) * cos((pow4(x) ^ sqrt(x)))) * cos(((pow6(x) * 3) ^ sqrt(x)))). Среднеквадратичная ошибка на 20 обучающих точках составила 0.00011744, корень из ошибки - 0.010837. Дерево содержит 18 узлов и имеет глубину 6. Полученная формула является приближением исходной функции; различие оценивается численно и показано на графике.

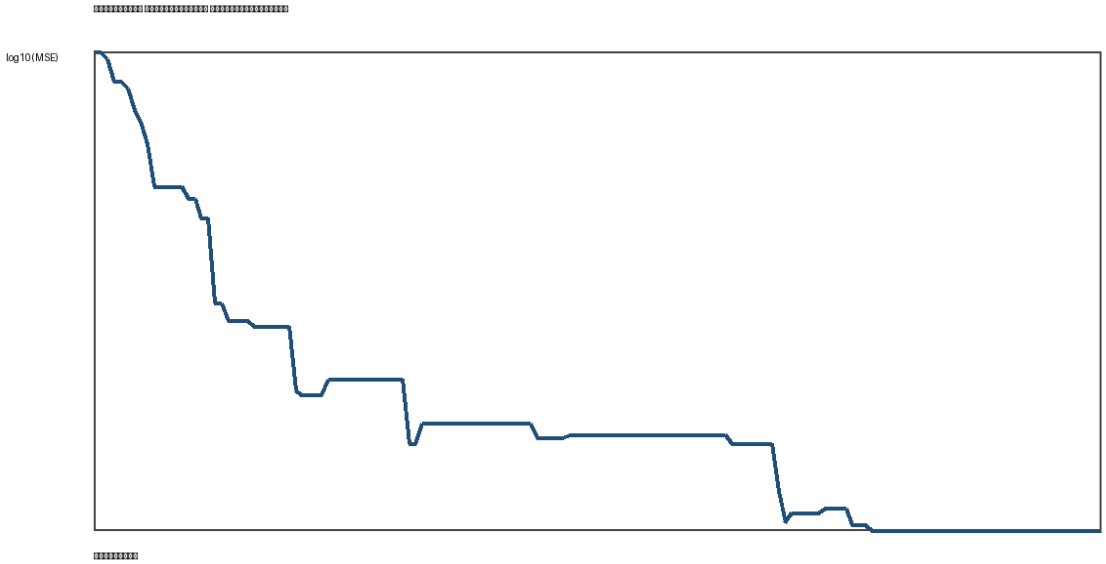


Рисунок 1 - Сходимость генетического программирования

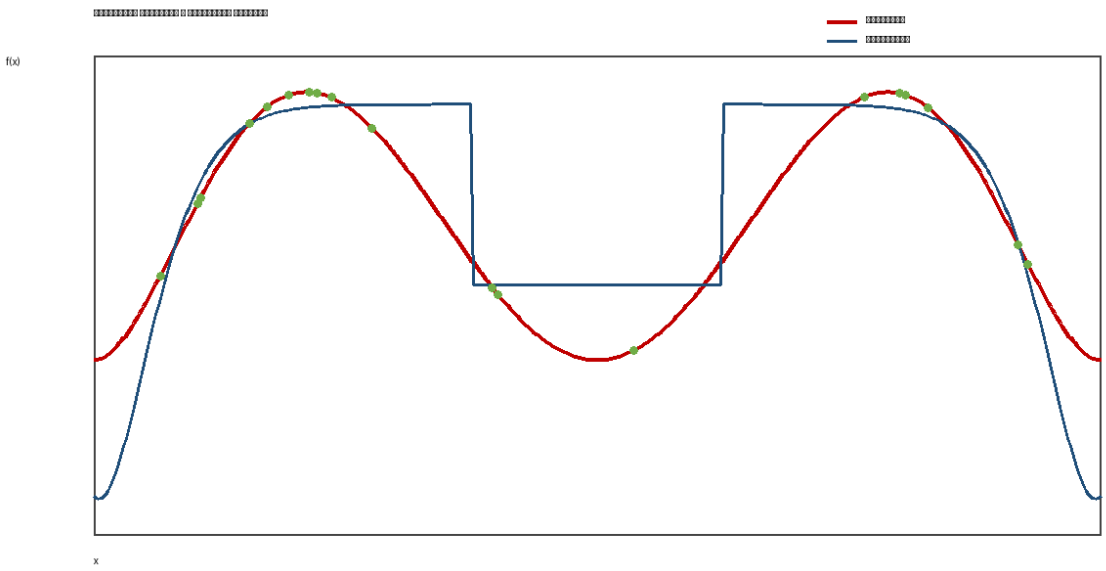


Рисунок 2 - Сравнение исходной и найденной функций

3.3 Графическое представление деревьев

Для контроля эволюции сохранены лучшие деревья поколений 0, 10, 25, 50, 100 и 150. На каждом узле показана операция или терминал, а заголовок содержит формулу соответствующей лучшей особи. Внутри программы дерево сохраняется для каждого поколения, а в отчет включена компактная сводная иллюстрация контрольных итераций.

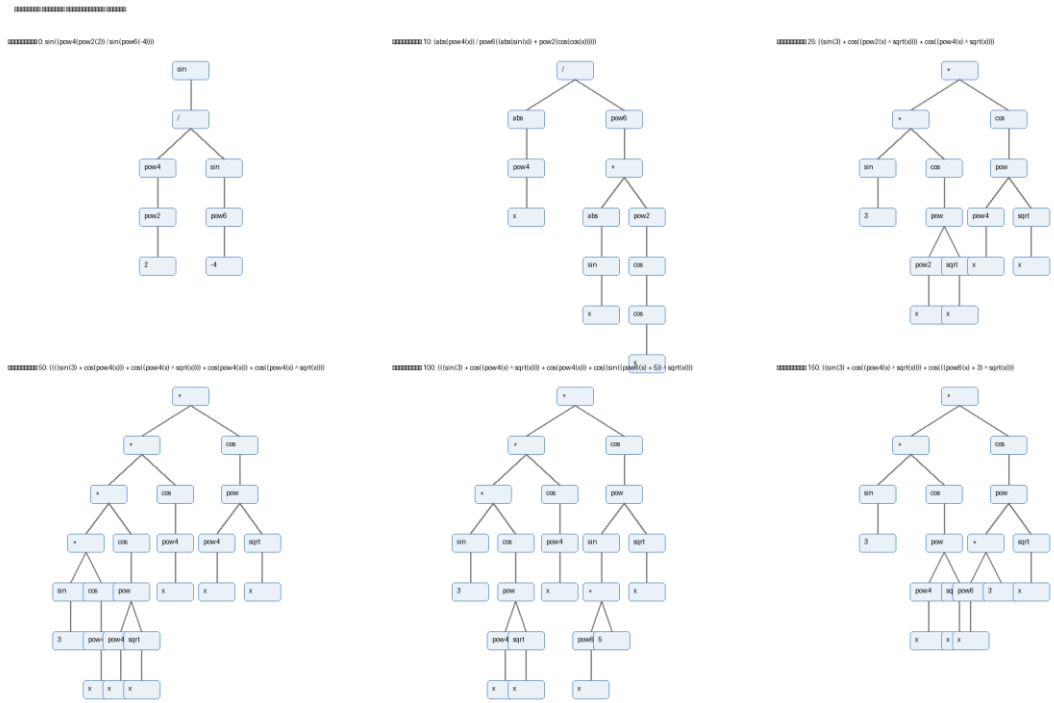


Рисунок 3 - Изменение лучшего программного дерева

3.4 Параметрическое исследование

Для исследования влияния параметров отдельно изменялись размер популяции, вероятность кроссовера и вероятность мутации. Остальные настройки сохранялись базовыми, а seed сдвигался на номер серии. Чем меньше MSE, тем ближе найденная программа к заданной функции.

Таблица 3 - Результаты параметрического исследования

Фактор	Значение	MSE	RMSE	Узлы	Время, с
population	250	0.000362	0.01903	8	4.221
population	750	0.000026	0.00510	14	23.362
crossover	0.60	0.000423	0.02058	6	6.851
crossover	1.00	0.000097	0.00985	20	28.891
mutation	0.05	0.000351	0.01874	15	17.140
mutation	0.25	0.000290	0.01703	11	15.768

Наиболее низкая ошибка среди серий получена при увеличенной популяции 750 особей. Уменьшение вероятности кроссовера ухудшило результат, поскольку обмен генетическим материалом происходил реже. Изменение вероятности мутации повлияло на баланс между сохранением удачных поддеревьев и исследованием новых структур.

4 ВЫВОДЫ

В работе реализован генетический алгоритм в режиме генетического программирования для символьной регрессии. Использовано древовидное представление программ, терминальное множество переменной x и целых констант от -5 до 5, а также функциональное множество арифметических и безопасных математических операций.

На 20 псевдоэкспериментальных точках функция была приближена с воспроизводимой среднеквадратичной ошибкой 0,00011744 в базовой серии. График сравнения показывает согласованное поведение найденной и исходной функций на области [-1, 1]. Параметрический эксперимент подтвердил, что увеличение популяции обычно повышает разнообразие поиска, но требует больше времени.

5 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС

Линейное представление программы - это хранение программы в виде последовательности терминалов и функций. Для однозначной интерпретации каждой функции задается арность. Наиболее распространены префиксная, инфиксная и постфиксная записи. В постфиксном варианте выражение читается слева направо с использованием стека: терминал помещается в стек, а операция извлекает из него нужное число аргументов и помещает результат обратно. Например, x x 2 pow4 * - можно интерпретировать как x - 2*x^4.

Преимущество линейного представления состоит в компактности, простоте сериализации и удобстве последовательного обхода. Недостаток заключается в необходимости контролировать корректность последовательности: после мутации может нарушиться баланс стека, арность операции или ограничение длины программы. В древовидном представлении структура корректна по построению, а обмен поддеревьями наглядно реализует кроссовер; линейная форма обычно проще для хранения, но сложнее для сохранения синтаксической корректности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа № 4. Генетическое программирование: методические указания.

2. Вирсански Э. Генетические алгоритмы на Python / пер. с англ. - М.: ДМК Пресс, 2020.

3. Скобцов Ю. А. Вычислительный интеллект. - СПб.: ГУАП, 2022.

4. GUAP. Нормативная документация для учебного процесса. - URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 15.10.2026).

Приложение А. Листинг алгоритма

```
"""Tree-based genetic programming for symbolic regression, laboratory work 4."""

from __future__ import annotations

import copy
import math
import random
from dataclasses import dataclass
from typing import Callable, Iterable



TARGET_FORMULA = "x^6 - 2*x^4 + x^2"



def target_function(x: float) -> float:
    return x**6 - 2.0 * x**4 + x**2



@dataclass
class Node:
    """A program tree node. Terminals have op='x' or op='const'."""

    op: str
    children: list["Node"]
    value: float | None = None

    def clone(self) -> "Node":
        return copy.deepcopy(self)

    def size(self) -> int:
        return 1 + sum(child.size() for child in self.children)

    def depth(self) -> int:
        return 1 if not self.children else 1 + max(child.depth() for child in self.children)



FUNCTION_ARITY: dict[str, int] = {
    "+": 2,
    "-": 2,
    "*": 2,
    "/": 2,
    "pow": 2,
    "abs": 1,
    "sin": 1,
    "cos": 1,
    "exp": 1,
    "sqrt": 1,
    "pow2": 1,
    "pow4": 1,
    "pow6": 1,
}



def protected_div(a: float, b: float) -> float:
    return a / b if abs(b) > 1e-9 else a



def safe_pow(a: float, b: float) -> float:
```

```
    if not math.isfinite(a) or not math.isfinite(b):
        return 0.0
    exponent = max(-6, min(6, int(round(b))))
    try:
        value = a**exponent
    except (OverflowError, ZeroDivisionError):
        return 0.0
    return value if math.isfinite(value) else 0.0



def evaluate(node: Node, x: float) -> float:
    if node.op == "x":
        return x
    if node.op == "const":
        return float(node.value or 0.0)
    values = [evaluate(child, x) for child in node.children]
    try:
        if node.op == "+":
            value = values[0] + values[1]
        elif node.op == "-":
            value = values[0] - values[1]
        elif node.op == "*":
            value = values[0] * values[1]
        elif node.op == "/":
            value = protected_div(values[0], values[1])
        elif node.op == "pow":
            value = safe_pow(values[0], values[1])
        elif node.op == "abs":
            value = abs(values[0])
        elif node.op == "sin":
            value = math.sin(values[0])
        elif node.op == "cos":
            value = math.cos(values[0])
        elif node.op == "exp":
            value = math.exp(max(-8.0, min(8.0, values[0])))
        elif node.op == "sqrt":
            value = math.sqrt(abs(values[0]))
        elif node.op == "pow2":
            value = values[0] ** 2
        elif node.op == "pow4":
            value = values[0] ** 4
        elif node.op == "pow6":
            value = values[0] ** 6
        else:
            raise ValueError(f"Unknown operation: {node.op}")
    except (OverflowError, ZeroDivisionError, ValueError):
        return 0.0
    return float(value) if math.isfinite(value) else 0.0



def expression(node: Node) -> str:
    if node.op == "x":
        return "x"
    if node.op == "const":
        value = float(node.value or 0.0)
        return str(int(value)) if value.is_integer() else f"{value:.3g}"
    if node.op in {"+", "-", "*", "/", "pow"}:
        symbol = "^" if node.op == "pow" else node.op
        return f"({expression(node.children[0])} {symbol} {expression(node.children[1])})"
    return f"{node.op}({expression(node.children[0])})"



def all_paths(node: Node, path: tuple[int, ...] = ()) -> list[tuple[int, ...]]:
    paths = [path]
    for index, child in enumerate(node.children):
        paths.extend(all_paths(child, path + (index,)))
    return paths
```

```
def get_subtree(node: Node, path: tuple[int, ...]) -> Node:
    current = node
    for index in path:
        current = current.children[index]
    return current



def replace_subtree(node: Node, path: tuple[int, ...], replacement: Node) -> Node:
    if not path:
        return replacement.clone()
    result = node.clone()
    parent = result
    for index in path[:-1]:
        parent = parent.children[index]
    parent.children[path[-1]] = replacement.clone()
    return result



class GeneticProgramming:
    """A compact generational GP implementation with tree crossover and mutation."""


    def __init__(
        self,
        x_values: Iterable[float],
        y_values: Iterable[float],
        population_size: int = 500,
        generations: int = 150,
        max_depth: int = 7,
        crossover_rate: float = 0.85,
        mutation_rate: float = 0.12,
        tournament_size: int = 5,
        parsimony: float = 1e-5,
        seed: int = 4,
    ):
        self.x_values = list(x_values)
        self.y_values = list(y_values)
        self.population_size = population_size
        self.generations = generations
        self.max_depth = max_depth
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.tournament_size = tournament_size
        self.parsimony = parsimony
        self.rng = random.Random(seed)
        self.history: list[dict[str, float | int | str]] = []
        self.snapshots: list[tuple[int, Node]] = []
        self.all_snapshots: list[tuple[int, Node]] = []


    def random_terminal(self) -> Node:
        if self.rng.random() < 0.65:
            return Node("x", [])
        return Node("const", [], float(self.rng.choice(list(range(-5, 6)))))


    def random_tree(self, depth: int, full: bool = False) -> Node:
        if depth <= 0 or (not full and depth > 1 and self.rng.random() < 0.35):
            return self.random_terminal()
        # The restricted power functions are equivalent to exponentiation by a
        # small integer and make the variant's polynomial representable compactly.
        weighted = ["+", "-", "*", "/", "pow", "abs", "sin", "cos", "exp", "sqrt", "pow2",
"pow4", "pow6"]
        op = self.rng.choice(weighted)
        return Node(op, [self.random_tree(depth - 1, full) for _ in range(FUNCTION_ARITY[op])])


    def initial_population(self) -> list[Node]:
        return [self.random_tree(self.rng.randint(2, self.max_depth), i % 2 == 0) for i in
range(self.population_size)]
```

```
def mse(self, node: Node) -> float:
    errors = []
    for x, y in zip(self.x_values, self.y_values):
        predicted = evaluate(node, x)
        if not math.isfinite(predicted) or abs(predicted) > 1e6:
            return 1e6
        errors.append((predicted - y) ** 2)
    return sum(errors) / len(errors)


def score(self, node: Node) -> float:
    return self.mse(node) + self.parsimony * node.size()


def select(self, population: list[Node], scores: list[float]) -> Node:
    indices = [self.rng.randrange(len(population)) for _ in range(self.tournament_size)]
    return population[min(indices, key=lambda index: scores[index])]


def mutate(self, node: Node) -> Node:
    paths = all_paths(node)
    path = self.rng.choice(paths)
    replacement_depth = max(0, self.max_depth - len(path) - 1)
    replacement = self.random_tree(self.rng.randint(0, replacement_depth), full=False)
    return replace_subtree(node, path, replacement)


def crossover(self, first: Node, second: Node) -> tuple[Node, Node]:
    first_path = self.rng.choice(all_paths(first))
    second_path = self.rng.choice(all_paths(second))
    left = replace_subtree(first, first_path, get_subtree(second, second_path))
    right = replace_subtree(second, second_path, get_subtree(first, first_path))
    if left.depth() > self.max_depth:
        left = first.clone()
    if right.depth() > self.max_depth:
        right = second.clone()
    return left, right


def run(self) -> tuple[Node, float]:
    population = self.initial_population()
    best = min(population, key=self.score).clone()
    for generation in range(self.generations + 1):
        scores = [self.score(individual) for individual in population]
        generation_best_index = min(range(len(population)), key=lambda i: scores[i])
        generation_best = population[generation_best_index]
        if self.score(generation_best) < self.score(best):
            best = generation_best.clone()
        best_mse = self.mse(best)
        self.history.append({
            "generation": generation,
            "mse": best_mse,
            "score": self.score(best),
            "size": best.size(),
            "depth": best.depth(),
            "formula": expression(best),
        })
        self.all_snapshots.append((generation, best.clone()))
        if generation in {0, 10, 25, 50, 100, self.generations}:
            self.snapshots.append((generation, best.clone()))
        if generation == self.generations:
            break
        next_population = [best.clone()]
        while len(next_population) < self.population_size:
            first = self.select(population, scores).clone()
            second = self.select(population, scores).clone()
            if self.rng.random() < self.crossover_rate:
                first, second = self.crossover(first, second)
            if self.rng.random() < self.mutation_rate:
                first = self.mutate(first)
            if self.rng.random() < self.mutation_rate:
                second = self.mutate(second)
            next_population.extend((first, second))
```

```
            population = next_population[: self.population_size]
        return best, self.mse(best)


def make_training_data(seed: int = 4) -> tuple[list[float], list[float]]:
    rng = random.Random(seed)
    x_values = sorted(rng.uniform(-1.0, 1.0) for _ in range(20))
    return x_values, [target_function(x) for x in x_values]
```

Приложение Б. Сценарий экспериментов

```
"""Run reproducible GP experiments and create report figures."""


from __future__ import annotations


import csv
import json
import math
import time
from pathlib import Path


from PIL import Image, ImageDraw, ImageFont


from gp_symbolic_regression import GeneticProgramming, evaluate, expression, make_training_data,
target_function



ROOT = Path(__file__).resolve().parent
RESULTS = ROOT / "results"
RESULTS.mkdir(exist_ok=True)



def write_csv(path: Path, rows: list[dict]) -> None:
    with path.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)



def plot_convergence(history: list[dict], path: Path) -> None:
    canvas = Image.new("RGB", (1200, 650), "white")
    draw = ImageDraw.Draw(canvas)
    left, top, right, bottom = 100, 70, 1130, 560
    draw.rectangle((left, top, right, bottom), outline="#444444", width=2)
    values = [max(float(row["mse"]), 1e-12) for row in history]
    logs = [math.log10(value) for value in values]
    lo, hi = min(logs), max(logs)
    if abs(hi - lo) < 1e-9:
        hi = lo + 1
    points = []
    for row, value in zip(history, logs):
        px = left + (right - left) * row["generation"] / max(1, history[-1]["generation"])
        py = bottom - (bottom - top) * (value - lo) / (hi - lo)
        points.append((px, py))
    draw.line(points, fill="#1f4e79", width=4)
    draw.text((left, 20), "Сходимость генетического программирования", fill="black")
    draw.text((left, 580), "Поколение", fill="black")
    draw.text((10, top), "log10(MSE)", fill="black")
    canvas.save(path)



def draw_tree(node, ax, x: float, y: float, dx: float, level: int = 0, parent=None) -> None:
    label = node.op if node.op != "const" else str(int(node.value or 0))
    if node.op == "x":
        label = "x"
    draw = ax[0]
    draw.rounded_rectangle((x - 28, y - 14, x + 28, y + 14), radius=5, fill="#eaf2f8",
outline="#4f81bd")
    draw.text((x - 20, y - 8), label, fill="black")
    if parent is not None:
        draw.line((parent[0], parent[1] + 14, x, y - 14), fill="#777777", width=2)
    if node.children:
        width = dx * (len(node.children) - 1)
```

```
        for index, child in enumerate(node.children):
            child_x = x - width / 2 + index * dx
            draw_tree(child, ax, child_x, y + 75, dx * 0.68, level + 1, (x, y))



def plot_tree(node, generation: int, path: Path) -> None:
    canvas = Image.new("RGB", (1400, 700), "white")
    draw = ImageDraw.Draw(canvas)
    draw.text((30,   20),   f"Лучшее   дерево,   поколение   {generation}:   {expression(node)}",
fill="black")
    draw_tree(node, (draw, canvas), 700, 90, 310)
    canvas.save(path)



def plot_tree_progress(snapshots, path: Path) -> None:
    canvas = Image.new("RGB", (1800, 1100), "white")
    draw = ImageDraw.Draw(canvas)
    draw.text((30, 20), "Эволюция лучшего программного дерева", fill="black")
    for index, (generation, node) in enumerate(snapshots):
        x0 = 20 + (index % 3) * 590
        y0 = 70 + (index // 3) * 500
        draw.text((x0, y0), f"Поколение {generation}: {expression(node)}", fill="black")
        draw_tree(node, (draw, canvas), x0 + 280, y0 + 50, 150)
    canvas.save(path)



def plot_comparison(best, x_values, y_values, path: Path) -> None:
    grid = [-1.0 + 2.0 * i / 499 for i in range(500)]
    target = [target_function(float(x)) for x in grid]
    predicted = [evaluate(best, float(x)) for x in grid]
    canvas = Image.new("RGB", (1200, 650), "white")
    draw = ImageDraw.Draw(canvas)
    left, top, right, bottom = 100, 70, 1130, 560
    draw.rectangle((left, top, right, bottom), outline="#444444", width=2)
    ymin = min(target + predicted) - 0.02
    ymax = max(target + predicted) + 0.02
    def point(x, y):
        return (left + (right-left) * (x + 1) / 2, bottom - (bottom-top) * (y-ymin)/(ymax-ymin))
    draw.line([point(x, y) for x, y in zip(grid, target)], fill="#c00000", width=4)
    draw.line([point(x, y) for x, y in zip(grid, predicted)], fill="#1f4e79", width=3)
    for x, y in zip(x_values, y_values):
        px, py = point(x, y)
        draw.ellipse((px-4, py-4, px+4, py+4), fill="#70ad47")
    draw.text((left, 20), "Сравнение исходной и найденной функций", fill="black")
    draw.text((left, 580), "x", fill="black")
    draw.text((10, top), "f(x)", fill="black")
    draw.line((850, 35, 880, 35), fill="#c00000", width=4)
    draw.text((890, 27), "исходная", fill="black")
    draw.line((850, 55, 880, 55), fill="#1f4e79", width=3)
    draw.text((890, 47), "найденная", fill="black")
    canvas.save(path)



def main() -> None:
    x_values, y_values = make_training_data()
    configs = {
        "baseline": dict(population_size=500, generations=150, max_depth=7, crossover_rate=0.85,
mutation_rate=0.12),
        "population_250":      dict(population_size=250,      generations=150,      max_depth=7,
crossover_rate=0.85, mutation_rate=0.12),
        "population_750":      dict(population_size=750,      generations=150,      max_depth=7,
crossover_rate=0.85, mutation_rate=0.12),
        "crossover_0.60":      dict(population_size=500,      generations=150,      max_depth=7,
crossover_rate=0.60, mutation_rate=0.12),
        "crossover_1.00":      dict(population_size=500,      generations=150,      max_depth=7,
crossover_rate=1.00, mutation_rate=0.12),
        "mutation_0.05":      dict(population_size=500,      generations=150,      max_depth=7,
crossover_rate=0.85, mutation_rate=0.05),
```

```
        "mutation_0.25":      dict(population_size=500,      generations=150,      max_depth=7,
crossover_rate=0.85, mutation_rate=0.25),
    }
    summaries = {}
    baseline = None
    study_rows = []
    for index, (label, params) in enumerate(configs.items()):
        started = time.perf_counter()
        gp = GeneticProgramming(x_values, y_values, seed=4 + index, **params)
        best, mse = gp.run()
        elapsed = time.perf_counter() - started
        result = {
            "label": label,
            "mse": mse,
            "rmse": mse**0.5,
            "size": best.size(),
            "depth": best.depth(),
            "formula": expression(best),
            "elapsed_seconds": elapsed,
            "parameters": params | {"seed": 4 + index, "tournament_size": 5, "parsimony": 1e-5},
        }
        summaries[label] = result
        if label == "baseline":
            baseline = (gp, best, result)
        if label != "baseline":
            factor, value = label.rsplit("_", 1)
            study_rows.append({"factor": factor, "value": value, "mse": f"{mse:.10g}", "rmse":
f"{mse**0.5:.10g}", "size": best.size(), "time_seconds": f"{elapsed:.3f}"})


    assert baseline is not None
    gp, best, base_result = baseline
    plot_convergence(gp.history, RESULTS / "convergence.png")
    plot_comparison(best, x_values, y_values, RESULTS / "function_comparison.png")
    plot_tree_progress(gp.snapshots, RESULTS / "tree_progress.png")
    for generation, node in gp.all_snapshots:
        plot_tree(node, generation, RESULTS / f"tree_generation_{generation:03d}.png")
    with (RESULTS / "training_data.csv").open("w", encoding="utf-8", newline="") as stream:
        writer = csv.writer(stream)
        writer.writerow(["x", "y"])
        writer.writerows(zip(x_values, y_values))
    write_csv(RESULTS / "parameter_study.csv", study_rows)
    summary = {
        "variant": 4,
        "target_formula": "f(x) = x^6 - 2*x^4 + x^2",
        "domain": "[-1, 1]",
        "points": 20,
        "terminal_set": ["x", "-5", "-4", "-3", "-2", "-1", "0", "1", "2", "3", "4", "5"],
        "function_set": ["+", "-", "*", "/", "protected pow", "abs", "sin", "cos", "exp", "sqrt",
"pow2", "pow4", "pow6"],
        "problems": summaries,
        "baseline_history": gp.history,
    }
    (RESULTS    /    "results_summary.json").write_text(json.dumps(summary,    ensure_ascii=False,
indent=2), encoding="utf-8")
    print(json.dumps(base_result, ensure_ascii=False, indent=2))



if __name__ == "__main__":
    main()
```